Service Mesh

move networking out of the code

Watch the 4-min video

In one line: a service mesh puts a proxy (the data plane) beside every service instance and configures all of them centrally (the control plane), giving every service mTLS identity, retries, timeouts, traffic splitting and uniform telemetry without code changes, at the cost of extra latency, resources and operational complexity.

1 The same library in five languages

A company runs two hundred services in Java, Go, Python and Node. Security wants every connection encrypted. Operations wants consistent retries, timeouts and metrics. Asking every team to add and upgrade the same libraries in every language never quite works: versions drift and some teams never finish. A service mesh moves these concerns into infrastructure.

2 Two parts

- **Data plane:** a small proxy, usually **Envoy** (Linkerd uses its own Rust proxy), runs as a **sidecar** next to each instance (lesson 28). Traffic in and out of the app is redirected through it (on Kubernetes, via iptables rules).
- **Control plane:** e.g. Istio's `istiod`. It turns operators' policies into proxy config, pushes it out, and acts as the certificate authority.

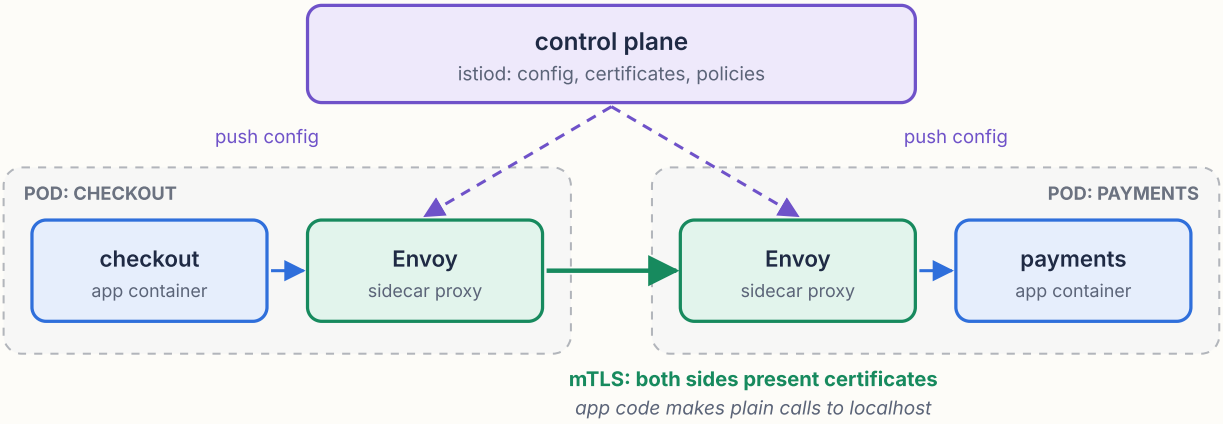


Figure 1. Every pod gets a sidecar proxy; all traffic flows proxy to proxy. The control plane configures every proxy and issues certificates.

3 What you get

Feature	What the proxies do
Security	issue each workload an identity certificate (SPIFFE IDs in Istio), encrypt all traffic with **mutual TLS**, rotate certs automatically, enforce "who may call whom"
Traffic management	timeouts, retries, load balancing, outlier ejection, canary and blue-green splits by weight or header
Resilience testing	inject delays or errors for a share of requests
Telemetry	request rate, error rate and latency for every service pair; propagate tracing headers (apps must still forward them)

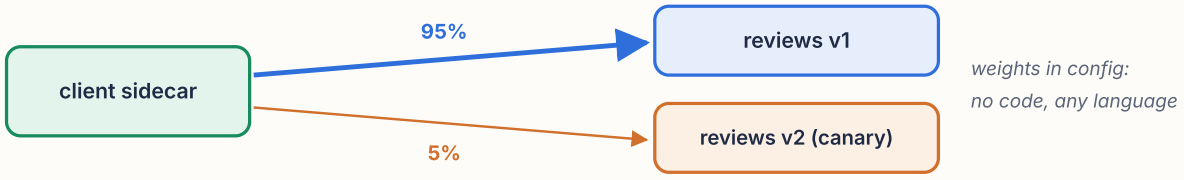


Figure 2. A canary release as configuration: the client's proxy sends 5% of requests to the new version.

All of this is configuration, identical across languages, audited in one place. Teams stop re-implementing retry logic, and dashboards for every service come almost for free.

4 What it costs

TRADE-OFFS

- **Latency:** two extra proxy hops per call (client and server side), typically around a millisecond or less each, but it adds up on long chains.
- **Resources:** CPU and memory for a proxy in every pod; at hundreds of pods this is real money.
- **Complexity:** the control plane must be run, upgraded and understood; misconfigured policies cause outages.
- **Debugging:** another layer between services. With a handful of services, a library or the platform's own features are simpler.

Newer "sidecarless" modes (Istio ambient mode, Cilium) move part of the proxy work to per-node components to cut this overhead.

WORKED EXAMPLE: STRICT MTLS IN ISTIO

A company runs Istio on Kubernetes. A mesh-wide `PeerAuthentication` with `mtls.mode: STRICT` makes every sidecar reject plain-text traffic, so every connection is encrypted and both ends are authenticated. An `AuthorizationPolicy` on the payments namespace allows requests only from the checkout service account. Certificates are short-lived and rotate automatically. The security team reviews two YAML files instead of two hundred codebases.

5 Key terms

Data plane: the proxies that carry traffic

Control plane: configures proxies, issues certificates

Envoy: widely used open-source L7 proxy

Traffic splitting: send a weighted share to another version

mTLS: TLS where both client and server present certificates

Ambient / sidecarless: mesh without a proxy per pod

6 Interview questions

Q1. What problems does a service mesh solve? BASIC

Consistent mTLS and service identity, traffic control (timeouts, retries, load balancing, canary splits, fault injection) and uniform telemetry, all without application code changes.

Q2. What are the data plane and the control plane? BASIC

The data plane is the set of proxies beside each service that carry traffic; the control plane configures them and issues certificates.

Q3. Security requires all service-to-service traffic encrypted and only checkout may call payments. How? SCENARIO

Enable strict mTLS mesh-wide so plain text is rejected, and add an authorisation policy allowing only checkout's identity to call payments.

Q4. You have 6 services in one language. Do you need a mesh? SCENARIO

Probably not: a shared library plus your platform's load balancing and TLS is simpler; adopt a mesh when service count, languages and security needs justify the overhead.

Q5. What does a service mesh cost? TRICKY

Extra latency and CPU/memory for every proxy, plus a complex control plane to operate, upgrade and debug.

Q6. Does a mesh give you distributed tracing for free? TRICKY

Partly: proxies emit spans, but the application must still forward trace headers from incoming to outgoing requests, or the spans won't join into one trace.

Next: SD-L-010 · Sidecar and Ambassador backendroadmap.com